

FINAL TECHNICAL ENGINEERING DISSERTATION

COURSIFY

An Enterprise-Grade Full-Stack Online Course Management & AI-Proctored E-Learning Platform

AUTHOR & DEVELOPER

Dhanush

PROJECT SCOPE

Full-Stack Web App

PRIMARY STACK

React 18 + Node + SQLite

ACADEMIC YEAR

2026

1. Introduction & Problem Statement

1.1 Background & Motivation

The digital transformation of education has accelerated the adoption of Learning Management Systems (LMS). However, the vast majority of existing web-based platforms suffer from two fundamental engineering and pedagogical flaws:

- 1. Passive Content Skimming & Skipping:** Students often navigate directly to the final video module or exam without completing foundational modules, severely degrading retention.
- 2. Compromised Certification Integrity:** Online evaluations are notoriously vulnerable to unauthorized tab switching, web searching, clipboard copy-pasting, and unattended sessions.

1.2 The Coursify Solution

Coursify is an engineered full-stack software application built to eliminate these vulnerabilities. It introduces a comprehensive learning suite combining:

- **107 Seeded Industry Courses** spanning 8 specialized computer science domains.

- **Sequential Progression Enforcement:** Programmatic module locking ensuring prerequisite mastery.
- **AI-Proctored Anti-Cheating Suite:** Real-time webcam monitoring, 3-strike tab-switch/window blur detection with procedural audio alarms, and 100% clipboard interceptors.
- **In-Lesson Study Scratchpad & Concept Quizzes:** Auto-saving relational markdown notes and immediate formative multiple-choice assessments.
- **5 Dynamic GPU-Accelerated Themes:** Instant token-driven theme transitions across Midnight Cyber, Matrix Emerald, Cosmic Amethyst, Oceanic Sapphire, and Sunset Ember.

2. System Architecture & Technical Flowcharts

2.1 High-Level 3-Tier Layered Architecture

The system is designed following a clean 3-Tier Client-Server-Database architecture with stateless JWT authentication:

```
+=====+ |
CLIENT TIER | | React 18 SPA (Vite) | 5 Dynamic CSS Theme Engines | Web Audio API Alerts
| | - Course Catalog & Search - Interactive Video Classroom (Sequential Locks) | | -
Student & Admin Dashboards - AI-Proctored Examination Booth |
+=====+=====+ |
| (HTTPS / REST JSON API / Bearer JWT) v
+=====+=====+ |
SERVER TIER | | Node.js (v20+) & Express.js REST API Server | | - Authentication
Middleware (JWT + Salted Bcrypt Hashing) | | - Role-Based Access Control (RBAC:
'student' vs 'admin') | | - Course Controller (107 Courses, Category Filters, Bookmarks)
| | - Enrollment & Progress Engine (Sequential Locks, SQLite Notes) | | - Exam
Proctoring Controller (535 Questions, 3-Strike Malpractice Evaluation) | | - Static
Single-Port Production Web Server |
+=====+=====+ |
| (Better-SQLite3 Prepared Statements) v
+=====+=====+ |
DATABASE TIER | | SQLite3 Relational Database (3NF Normalized with Foreign Key Cascade
Deletions) | | [users] [courses] [lessons] [enrollments] [lesson_progress] | |
[lesson_notes] [bookmarks] [exam_questions] [exam_submissions] |
+=====+=====+
```

2.2 AI-Proctored Anti-Cheating & Malpractice State Machine Flowchart

The examination suite executes a strict client-server validation state machine:

```
[ Student Attains 100% Course Module Progress ] | v [ Open AI-Proctor Exam Room ] | v [
Request Camera & Microphone (getUserMedia API) ] | v [ Mount Live Video Feed + Activate
Browser Event Listeners: ] [ - document.addEventListener('visibilitychange') ] [ -
window.addEventListener('blur') ] [ - document.addEventListener('copy', 'paste', 'cut')
] [ - document.addEventListener('contextmenu') ] | +-----+-----+
+-----+ | | (Tab Switch / Blur Event) (Valid MCQ Selections) | | v v [ Beep Alarm Tone
] [ Submit Answer Payload ] [ Increment Strike ] | | v +-----+-----+ [ Backend
Controller ] | | [ - Verify 100% Learn] (Strikes < 3) (Strikes = 3) [ - Calculate Score
%] | | | v v +-----+-----+ [ Warning Modal] [ AUTO-TERMINATE & ] | | [ Return Exam ] [
DISQUALIFY EXAM ] (Score ≥ 60%) (Score < 60%) | | v v [ Issue Cert ] [ Fail Attempt ]
```

3. Relational Database Design & Normalization (3NF)

The database utilizes 9 relational tables structured in **Third Normal Form (3NF)**. Every table is free of transitive dependencies and maintains strict referential integrity with foreign key cascade actions:

Table Name	Primary Key	Foreign Keys	Purpose & Stored Fields
users	id	None	User credentials (email, bcrypt password hash), name, role (student / admin), timestamps.
courses	id	instructor_id → users(id)	Course metadata: title, description, category, level, duration, thumbnail URL, price, rating.
lessons	id	course_id → courses(id)	Course video modules: title, video_url, markdown content, duration, order_index.
enrollments	id	user_id → users(id) course_id → courses(id)	Student registration records, completion status boolean, enrolled timestamp.
lesson_progress	id	user_id → users(id) lesson_id → lessons(id)	Individual module completion checklist status with completed timestamps.
lesson_notes	id	user_id → users(id) lesson_id → lessons(id)	Personal study scratchpad markdown notes auto-saved per student per module.
bookmarks	id	user_id → users(id) course_id → courses(id)	Student wishlist bookmark records with creation timestamps.
exam_questions	id	course_id → courses(id)	535 seeded certification questions: question text, options A/B/C/D, correct option key, explanation.
exam_submissions	id	user_id → users(id) course_id → courses(id)	Final exam attempt records: score percentage, passed flag, malpractice violations count, certificate code.

4. Tools, Libraries & Technology Stack Justification

Why this stack was chosen: React 18 and Vite deliver instantaneous client-side navigation without bulky overhead. Node.js and Express provide an asynchronous, high-concurrency RESTful API. Better-SQLite3 provides microsecond synchronous SQL query execution without external database process dependencies.

Tool / Technology	Category	Role in Project
React 18.3	Frontend Framework	Stateful single-page application with modular component hierarchy.
Vite 5.3	Build Engine	Fast development hot module replacement (HMR) and optimized tree-shaken production bundles.
Better-SQLite3	Database Engine	C++ SQLite bindings executing prepared SQL statements directly inside Node memory.
JSONWebToken	Security / Auth	Signs and verifies cryptographic stateless authentication tokens passed via <code>Authorization: Bearer </code> .
Bcrypt.js	Password Hashing	One-way adaptive cryptographic hashing with 10 salt rounds protecting user passwords.
Web Audio API	Browser API	Procedural dual-tone warning sound generation on tab-switch violations without external MP3 files.
Page Visibility API	Browser API	Real-time tracking of browser window focus, tab visibility, and application defocusing.
Lucide React	Iconography	Lightweight, scalable vector icons across all pages and UI controls.
Canvas Confetti	UX Animation	GPU-rendered particle fireworks triggered upon 100% course completion.

5. Detailed Step-by-Step Build Journey from Scratch

Step 1: Monorepo Architecture & Package Management

Created a unified workspace housing both the `client/` (React Vite) and `server/` (Express) codebases. Configured root `package.json` scripts enabling concurrent local execution (`npm run dev`) and automated cloud building (`npm run build`).

Step 2: Database Schema & 107-Course Seeding

Engineered the database schema in `server/src/config/db.js`. Designed automated seed scripts (`seed_100_courses.js` and `seed_exam_questions.js`) that populated 107 courses across 8 domains and 535 certification questions.

Step 3: Secure Express REST API Layer

Built 17 RESTful endpoints. Implemented authentication middleware (`auth.js`) that extracts bearer tokens, validates signatures, and enforces Role-Based Access Control (RBAC). Admin endpoints respond with `403 Forbidden` when accessed by standard student accounts.

Step 4: Interactive Video Classroom & Sequential Progression

Constructed the `LearningRoom.jsx` classroom. Integrated:

- **Sequential Progression Algorithm:** Calculated dynamically from completed module IDs.
- **Pre-Content & Prerequisites Module:** Displays toolchain requirements (JDK, VS Code, Git) and course expectations.
- **Personal Study Scratchpad:** Auto-saves code snippets and notes to the database.
- **Concept Check Quizzes:** Instant multiple-choice tests per module.

Step 5: AI-Proctored Anti-Cheating Examination Suite

Engineered `ExamProctorRoom.jsx`. Integrated `navigator.mediaDevices.getUserMedia` for camera monitoring, attached event interceptors for `visibilitychange`, `blur`, `copy`, `paste`, and shortcut keys, and connected the 3-strike malpractice disqualification policy.

Step 6: 5 Dynamic Theme Palettes & Local Storage Persistence

Constructed an ultra-fast theme engine in `index.css` using CSS custom properties mapped under `data-theme` selectors. Designed a top navigation palette dropdown that stores preferences in `localStorage`.

Step 7: Automated Test Suite & Cloud Deployment

Developed an end-to-end audit script `server/test_all_endpoints.js` that verifies all 17 endpoints. Configured Express static file serving for single-port deployment on cloud platforms like Render.

6. Core Code Implementations & Technical Analysis

6.1 AI-Proctoring Sensor & Anti-Cheating Engine

```
// client/src/pages/ExamProctorRoom.jsx
useEffect(() => {
  if (!examStarted || examSubmitted) return;

  const handleVisibilityChange = () => {
    if (document.hidden) {
      triggerMalpracticeStrike('Tab switch detected! You navigated away from the exam window.');
```

6.2 Sequential Learning Lock Check

```
// client/src/pages/LearningRoom.jsx
const isLessonUnlocked = (index) => {
  if (index === 0) return true; // First lesson is always unlocked
  const prevLesson = lessons[index - 1];
  return prevLesson ? completedLessonIds.has(prevLesson.id) : false;
};
```

6.3 JWT Authentication & RBAC Middleware

```
// server/src/middleware/auth.js
const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ success: false, message: 'Access denied. No token provided.' });
  }
  try {
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret');
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Invalid or expired token.' });
  }
};

const authorize = (roles = []) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: 'Forbidden. Insufficient permissions.' });
  }
  next();
};
```

7. Terminal Commands & API Audit Verification

7.1 Project Execution Commands

Task	Command	Description
Install Dependencies	<code>npm run install:all</code>	Installs root, backend, and frontend packages.
Launch Dev Server	<code>npm run dev</code>	Launches Express (port 5000) & React Vite (port 3000) concurrently.
Run Automated Audit	<code>node server/test_all_endpoints.js</code>	Runs automated HTTP assertions across all 17 system endpoints.
Re-seed Database	<code>npm run seed</code>	Seeds 107 courses and 535 certification exam questions.
Build for Production	<code>npm run build</code>	Compiles React Vite client into production bundle.
Start Production Server	<code>npm start</code>	Launches Express server bound to <code>0.0.0.0</code> .

7.2 17-Endpoint Automated Verification Table

#	Method	Endpoint	Access	Description & Status
1	GET	/api/health	Public	Server health & timestamp (200 OK)
2	POST	/api/auth/register	Public	Student account creation (201 Created)
3	POST	/api/auth/login	Public	Student/Admin JWT authentication (200 OK)
4	GET	/api/auth/me	Authenticated	Current session verification (200 OK)
5	GET	/api/courses/categories	Public	8 domain categories listing (200 OK)
6	GET	/api/courses	Public	Catalog search & filter queries (200 OK)
7	GET	/api/courses/:id	Public	Course syllabus & details (200 OK)
8	POST	/api/courses/:id/bookmark	Student	Wishlist heart bookmark toggle (200 OK)
9	GET	/api/courses/my/bookmarks	Student	User saved bookmarks list (200 OK)

#	Method	Endpoint	Access	Description & Status
10	POST	/api/enrollments	Student	Enroll in selected course (200 OK)
11	GET	/api/enrollments/my	Student	Active & completed enrollments (200 OK)
12	GET	/api/enrollments/courses/:id/learn	Student	Classroom syllabus & progress (200 OK)
13	POST	/api/enrollments/courses/:id/lessons/:id/complete	Student	Lesson completion toggle (200 OK)
14	POST	/api/enrollments/lessons/:id/notes	Student	Save personal study notes (200 OK)
15	GET	/api/enrollments/lessons/:id/notes	Student	Fetch saved lesson notes (200 OK)
16	GET	/api/enrollments/courses/:id/exam	Student	Fetch 5 exam MCQs (200 OK)
17	POST	/api/enrollments/courses/:id/exam/submit	Student	Submit exam & issue certificate (200 OK)

8. Conclusion & Future Enhancements

The **Coursify** project successfully implements a modern, resilient, and human-crafted educational platform. By integrating sequential progression locking, in-lesson study scratchpads, 5 dynamic theme palettes, and an AI-proctored examination suite with 3-strike malpractice termination, it establishes a reliable paradigm for digital course delivery and authentic certification.

Future Roadmap Enhancements:

1. AI Facial Recognition for continuous biometrics verification during examinations.
2. Peer-to-peer real-time study discussion rooms using WebSockets.
3. Live screen sharing recording directly saved to cloud object storage (S3/GCS).

Coursify E-Learning Platform • Engineering Dissertation

Prepared and Submitted by **Dhanush** • 2026